



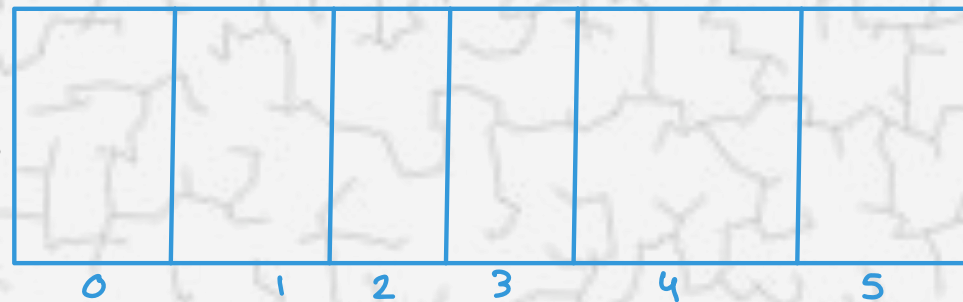
<http://algs4.cs.princeton.edu>

3.4 HASH TABLES → يعلمني وين اخزنه

- *hash functions*
- *separate chaining*
- *linear probing*
- *context*

7, 15, 6, 3

اذا بعطيه الـ (element) وبستعمل
المعادله علشان يعلمني وين اخزنه
 $f(x) = x \% 3 \text{ mod } 6$
لم ممكن تختلف



Uniform → اذا كل رقم بعطيه
(index) مختلف لان ما فيه تهادم

Collugin ← not uniform → بيكون اودري مخزنه
دجزي الـ index ويطلب مني اخزنها

Hashing: basic plan

Save items in a **key-indexed table** (index is a function of the key).

من مميز استخدامه
بطريقة معينة بحيث اول رقم الخانة الي الادات للشخص كامل مخزنه فيها

Hash function. Method for computing array index from key.

$\text{hash}^{\text{key}}(\text{"it"}) = 3$

$\text{hash}(\text{"times"}) = 3$

0	
1	
2	
3	"it"
4	
5	

Issues.

- Computing the hash function.
- Equality test: Method for checking whether two keys are equal. →
- Collision** resolution: Algorithm and data structure →
to handle two keys that hash to the same array index.

بيروحو لنفس الخانة

وحده راحت مجزت وجت الثانيه
تخذه لوقت المكان محجوز.

Classic space-time tradeoff.

- No space limitation: trivial hash function with key as index.
- No time limitation: trivial collision resolution with sequential search.
- Space and time limitations: hashing (the real world).

Maps



- ❑ A map models a searchable collection of key-value entries. مجموعه (Collection) یمن الاث →
عن حادیق (Value) یمنله > (Key).
- ❑ The main operations of a map are for searching, inserting, and deleting items ← ائنا صیرق
- ❑ Multiple entries with the same key are not allowed. ا.د (Key) ما یبکور
- ❑ Applications:
 - address book.
 - student-record database.



<http://algs4.cs.princeton.edu>

3.4 HASH TABLES

- *hash functions*
- *separate chaining*
- *linear probing*
- *context*

Computing the hash function

Idealistic goal. Scramble the keys uniformly to produce a table index.

- Efficiently computable.
- Each table index equally likely for each key.

thoroughly researched problem,
still problematic in practical applications



Ex 1. Phone numbers.

- Bad: first three digits.
- Better: last three digits.

Ex 2. Social Security numbers.

- Bad: first three digits.
- Better: last three digits.

573 = California, 574 = Alaska
(assigned in chronological order within geographic region)

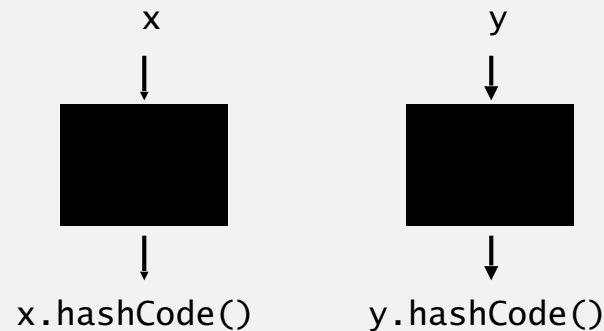
Practical challenge. Need different approach for each key type.

Java's hash code conventions

All Java classes inherit a method `hashCode()`, which returns a 32-bit `int`.

Requirement. If `x.equals(y)`, then `(x.hashCode() == y.hashCode())`.

Highly desirable. If `!x.equals(y)`, then `(x.hashCode() != y.hashCode())`.



Default implementation. Memory address of `x`.

Legal (but poor) implementation. Always return 17.

Customized implementations. `Integer`, `Double`, `String`, `File`, `URL`, `Date`, ...

User-defined types. Users are on their own.



Map ADT

- **get(k):** if the map M has an entry with key k, return its associated value; else, return null
← **بروح المظنه (Key) معين يرجع لي اذ (value) كامله ، و اذا المظنه (Key) غلط يرجع لي (null) .**
- **put(k, v):** insert entry (k, v) into the map M; if key k is not already in M, then return null; else, return old value
← **حط قفيه ، بمعني (Key) و (value) يروح يثبت اذ (Key) جالكان المظنه تبعه بيحطه ويحطه اذ (value) معه . ويرجعهم سواء .**
- **remove(k):** if the map M has an entry with key k, remove it from M and return its associated value; else, return null
← **بتلفي ، بتروح بتروح اذ (value) كقيه ويلغيها واذا موجوده يرجع لي (null) .**
- **size(), isEmpty()**
← **الشيء الذي يلف ما كل في المصنف**
← **فافيه اذ لا**
← **العدد**
- **entrySet():** return an iterable collection of the entries in M
← **بتروح لي كذا (collection of the entries) التي موجودة . التي عبارة عن (Key) و (value) .**
- **keySet():** return an iterable collection of the keys in M
← **بترجع لي (collection) التي بيوكس ويرجع لي اذ (Keys) .**
- **values():** return an iterator of the values in M
← **بترجع كل اذ (value) التي عنى .**

Example

Operation	Output	Map
isEmpty()	true	$\emptyset$
put(5,A)	null	$(5,A)$
put(7,B)	null	$(5,A),(7,B)$
put(2,C)	null	$(5,A),(7,B),(2,C)$
put(8,D)	null	$(5,A),(7,B),(2,C),(8,D)$
put(2,E)	C	$(5,A),(7,B),(2,E),(8,D)$
get(7)	B	$(5,A),(7,B),(2,E),(8,D)$
get(4)	null	$(5,A),(7,B),(2,E),(8,D)$
get(2)	E	$(5,A),(7,B),(2,E),(8,D)$
size()	4	$(5,A),(7,B),(2,E),(8,D)$
remove(5)	A	$(7,B),(2,E),(8,D)$
remove(2)	E	$(7,B),(8,D)$
get(2)	null	$(7,B),(8,D)$
isEmpty()	false	$(7,B),(8,D)$

← لأن الـ (2) موجودة فأخرج بدل على الحرف القديم وحل الجديد والقديم رجعة .

← دأخ يدور عليها ورجع لي الـ (Value) تبصا .

← دأخ دور عليها ما حملها رجع لي (null) .

← دأخ يدور عليها ويرجع قيمتها .

ويرجع الـ Value تبصا

ويرجع الـ Value تبصا

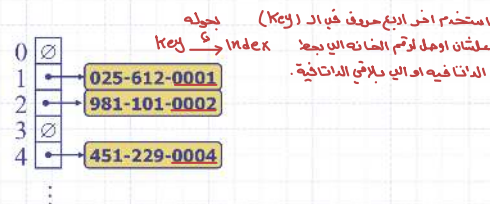
get(2) دور عليها مش موجوده رجع لي (null)

رجع لي isEmpty؟ كذا مش خافيه

✱ لمن بيدور على قيمه معيه ديشغل عليها : (1) ه

More General Kinds of Keys

- But what should we do if our keys are not integers in the range from 0 to $N - 1$?
 - Use a **hash function** to **map general keys to corresponding indices in a table**.
 - For instance, the last four digits of a Social Security number.



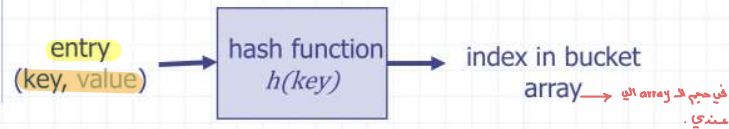
Hash Functions and Hash Tables



- A **hash function** h maps keys of a given type to integers in a fixed interval $[0, N - 1]$.
 - Example: اجبره بغير رقم
تحت الرقم تحت الارقام
$$h(x) = x \bmod N$$

is a hash function for integer keys
 - The integer $h(x)$ is called the **hash value** of key x . الرقم الي بيطلع معنى من الـ (Hash Value) يعنى الـ (Hash Pointer).
 - A **hash table** for a given key type consists of
 - Hash function h
 - Array (called table) of size N
- ← هو المرفقه الي بجزئ فيها

Hash Functions and Hash Tables



When implementing a map with a hash table, the goal is to store item (k, v) at index $i = h(k)$

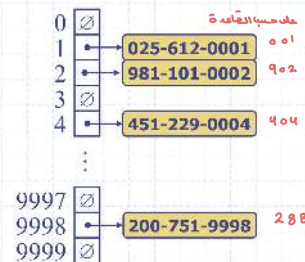
انفذ صدي hashmap على صدي
ال key يحول الى $h(k)$ يدخل ب index معين اسمه أ.

Example

- We design a hash table for a map storing entries as (SSN, Name), where SSN (social security number) is a nine-digit positive integer
- Our hash table uses an array of size $N = 10,000$ and the hash function $h(x) = \text{last four digits of } x$

عدد ال 9 ارقام مستساوي ونريد منه المينور بالذبحر .

0000 1235



Implementing hash code: integers, booleans, and doubles

Java library implementations

```
public final class Integer
{
    private final int value;
    ...

    public int hashCode()
    { return value; }
}
```

```
public final class Boolean
{
    private final boolean value;
    ...

    public int hashCode()
    {
        if (value) return 1231;
        else      return 1237;
    }
}
```

```
public final class Double
{
    private final double value;
    ...

    public int hashCode()
    {
        long bits = doubleToLongBits(value);
        return (int) (bits ^ (bits >>> 32));
    }
}
```

↑
convert to IEEE 64-bit representation;
xor most significant 32-bits
with least significant 32-bits

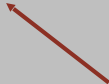
Warning: -0.0 and +0.0 have different hash codes

Implementing hash code: strings

Java library implementation

```
public final class String
{
    private final char[] s;
    ...

    public int hashCode()
    {
        int hash = 0;
        for (int i = 0; i < length(); i++)
            hash = s[i] + (31 * hash);
        return hash;
    }
}
```

 *i*th character of s

char	Unicode
...	...
'a'	97
'b'	98
'c'	99
...	...

- Horner's method to hash string of length L : L multiplies/adds.
- Equivalent to $h = s[0] \cdot 31^{L-1} + \dots + s[L-3] \cdot 31^2 + s[L-2] \cdot 31^1 + s[L-1] \cdot 31^0$.

Ex. `String s = "call";`
`int code = s.hashCode();` $\longleftarrow 3045982 = 99 \cdot 31^3 + 97 \cdot 31^2 + 108 \cdot 31^1 + 108 \cdot 31^0$
 $= 108 + 31 \cdot (108 + 31 \cdot (97 + 31 \cdot (99)))$
(Horner's method)

Implementing hash code: strings

Performance optimization.

- Cache the hash value in an instance variable.
- Return cached value.

```
public final class String
{
    private int hash = 0;
    private final char[] s;
    ...

    public int hashCode()
    {
        int h = hash;
        if (h != 0) return h;
        for (int i = 0; i < length(); i++)
            h = s[i] + (31 * h);
        hash = h;
        return h;
    }
}
```

← cache of hash code

← return cached value

← store cache of hash code

Q. What if hashCode() of string is 0?

Implementing hash code: user-defined types

```
public final class Transaction implements Comparable<Transaction>
{
    private final String  who;
    private final Date    when;
    private final double  amount;

    public Transaction(String who, Date when, double amount)
    { /* as before */ }

    ...

    public boolean equals(Object y)
    { /* as before */ }

    public int hashCode()
    {
        int hash = 17;
        hash = 31*hash + who.hashCode();
        hash = 31*hash + when.hashCode();
        hash = 31*hash + ((Double) amount).hashCode();
        return hash;
    }
}
```

nonzero constant

typically a small prime

for reference types,
use hashCode()

prime number *نِسْخَه*
for primitive types,
use hashCode()
of wrapper type

Hash code design

"Standard" recipe for user-defined types.

- Combine each significant field using the $31x + y$ rule.
- If field is a primitive type, use wrapper type `hashCode()`.
- If field is `null`, return 0. or reference (نظف لا hash code) or array (نطبق كل فيه بال array الي عندي type)
- If field is a reference type, use `hashCode()`. ← applies rule recursively
- If field is an array, apply to each entry. ← or use `Arrays.deepHashCode()`

In practice. Recipe works reasonably well; used in Java libraries.

In theory. Keys are bitstring; "universal" hash functions exist.

Basic rule. Need to use the whole key to compute hash code; consult an expert for state-of-the-art hash codes.

Modular hashing

Hash code. An int between -2^{31} and $2^{31} - 1$.

Hash function. An int between 0 and $M - 1$ (for use as array index).

typically a prime or power of 2

```
private int hash(Key key)
{ return key.hashCode() % M; }
```

bug

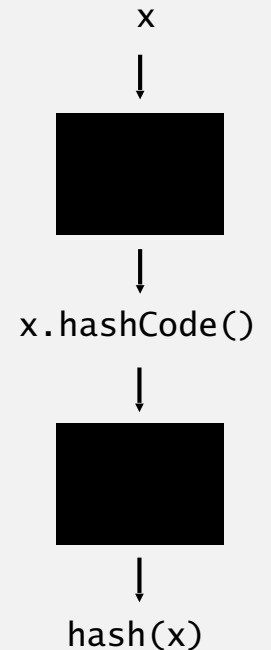
```
private int hash(Key key)
{ return Math.abs(key.hashCode()) % M; }
```

1-in-a-billion bug

hashCode() of "polygenelubricants" is -2^{31}

```
private int hash(Key key)
{ return (key.hashCode() & 0x7fffffff) % M; }
```

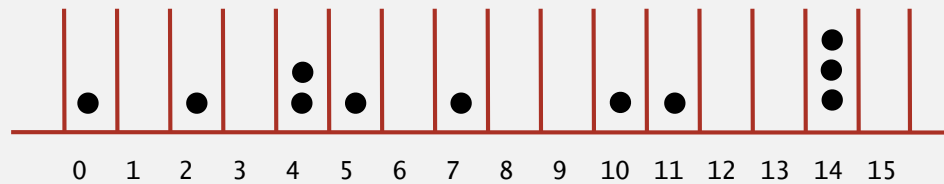
correct



Uniform hashing assumption

Uniform hashing assumption. Each key is equally likely to hash to an integer between 0 and $M - 1$.

Bins and balls. Throw balls uniformly at random into M bins.



Birthday problem. Expect two balls in the same bin after $\sim \sqrt{\pi M / 2}$ tosses.

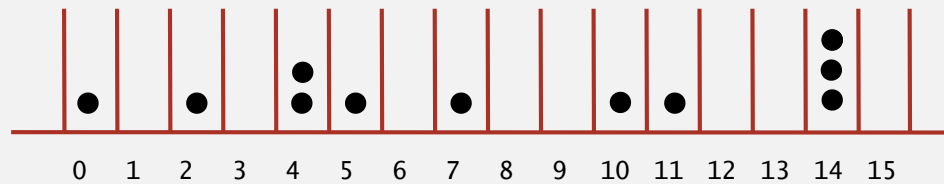
Coupon collector. Expect every bin has ≥ 1 ball after $\sim M \ln M$ tosses.

Load balancing. After M tosses, expect most loaded bin has $\Theta(\log M / \log \log M)$ balls.

Uniform hashing assumption

Uniform hashing assumption. Each key is equally likely to hash to an integer between 0 and $M - 1$.

Bins and balls. Throw balls uniformly at random into M bins.



Java's String data uniformly distribute the keys of Tale of Two Cities



<http://algs4.cs.princeton.edu>

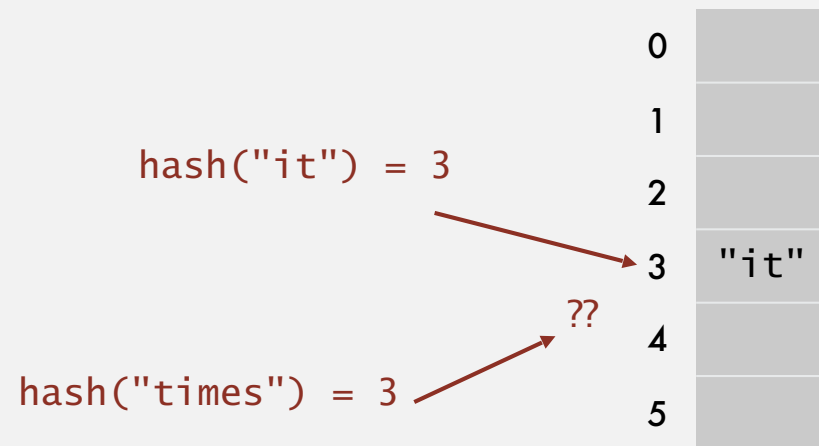
3.4 HASH TABLES

- *hash functions*
- *separate chaining*
- *linear probing* سلسلة
- *context*

Collisions

Collision. Two distinct keys hashing to same index.

- Birthday problem $\Rightarrow$ can't avoid collisions unless you have a ridiculous (quadratic) amount of memory.
- Coupon collector + load balancing $\Rightarrow$ collisions are evenly distributed.



Challenge. Deal with collisions efficiently.

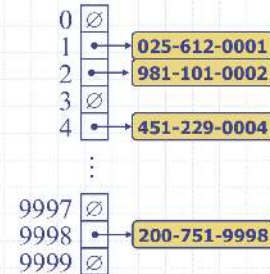
Example – Cont.

- What if we have a new entry to add to the bucket array:

(037-879-0002)

- $h(037-879-0002) = 0002 = 2$
- A **collision** has just occurred.

تصادف الرقم بـ (Index) (2) بلاقيها فل محجوزه ،
بسمية (collision) .
تصادف



Hash Tables

11

كيف نوجد mod ؟

أ- نخرج mod ونختار (4) جعدين نضرب القاعد هذي : $A \div B \times B$

ب- او نقسم A بـ B ونأخذ الرقم الصحيح بدون فواصل بعددين $A \div B \times B$
الخلاصة
من القيمة

Ex: $263 \mod 14$

$$263 \div 14 = 18 \rightarrow 263 - 18 \times 14 = 11$$

What is Hash function for following values to store them in array

	Bea	Tim	Leo	Sam	Mia	Zoe	Jan	Lou	Max	Ada	Ted
Index number	0	1	2	3	4	5	6	7	8	9	10
	sum ASCII codes					Mod		size of array			
Mia	M	77	i	105	a	97	279	4	1	0	5
Tim	T	84	i	105	m	109	298	1	0	5	6
Bea	B	66	e	101	a	97	264	5	0	5	9
Zoe	Z	90	e	101	e	101	302	6	0	5	2
Jan	J	74	a	97	n	110	281	9	0	5	7
Ada	A	65	d	100	a	97	262	2	0	5	8
Leo	L	76	e	101	o	111	288	3	0	5	10
Sam	S	83	a	97	m	109	289	7	0	5	
Lou	L	76	o	111	u	117	304	8	0	5	
Max	M	77	a	97	x	120	294	10	0	5	
Ted	T	84	e	101	d	100	285		0	5	

حجم array
Prime : 11
يكون عدد الكائنات
قليلا

جمع
كلهم

ما فيها تكرار

الذي يجب بالاول يحسن والي يجب بعده
هو ان عنده المتكامل لازم يحل مشكلته

ودائم اذا بنختار حجم الـ array نأخذ ارقام Prime

Link:- https://www.youtube.com/watch?v=KyUTuwz_b7Q

Hash Tables

12

If new value is getting the address which occupied then new value will be inserted in next available position

Mia	M	77	i	105	a	97	279	4
Tim	T	84	i	105	m	109	298	1
Bea	B	66	e	101	a	97	264	0
Zoe	Z	90	o	111	e	101	302	5
Sue	S	83	u	117	e	101	301	4

مكرر

Bea	Tim			Mia	Zoe					
0	1	2	3	4	5	6	7	8	9	10

Bea	Tim			Mia	Zoe					
0	1	2	3	4	5	6	7	8	9	10

Bea	Tim			Mia	Zoe					
0	1	2	3	4	5	6	7	8	9	10

Bea	Tim			Mia	Zoe	Sue				
0	1	2	3	4	5	6	7	8	9	10

© 2014 Goodrich, Tamassia, Godwasser Hash Tables 19

Solving collision by adding new value to next available position is called **open addressing** because every location is open to any item. Open addressing can use variety of open addressing techniques but this particular is called linear probing because if calculated address is occupied then linear search is used find next available slot

linear search

Mia	M	77	i	105	a	97	279	4
Tim	T	84	i	105	m	109	298	1
Bea	B	66	e	101	a	97	264	0
Zoe	Z	90	o	111	e	101	302	5
Sue	S	83	u	117	e	101	301	4
Len	L	76	e	101	n	110	287	1
Moe	M	77	o	111	e	101	289	3
Lou	L	76	o	111	u	117	304	7
Rae	R	82	a	97	e	101	280	5
Max	M	77	a	97	x	120	294	8
Tod	T	84	o	111	d	100	295	9

محمود فادحت الثاني

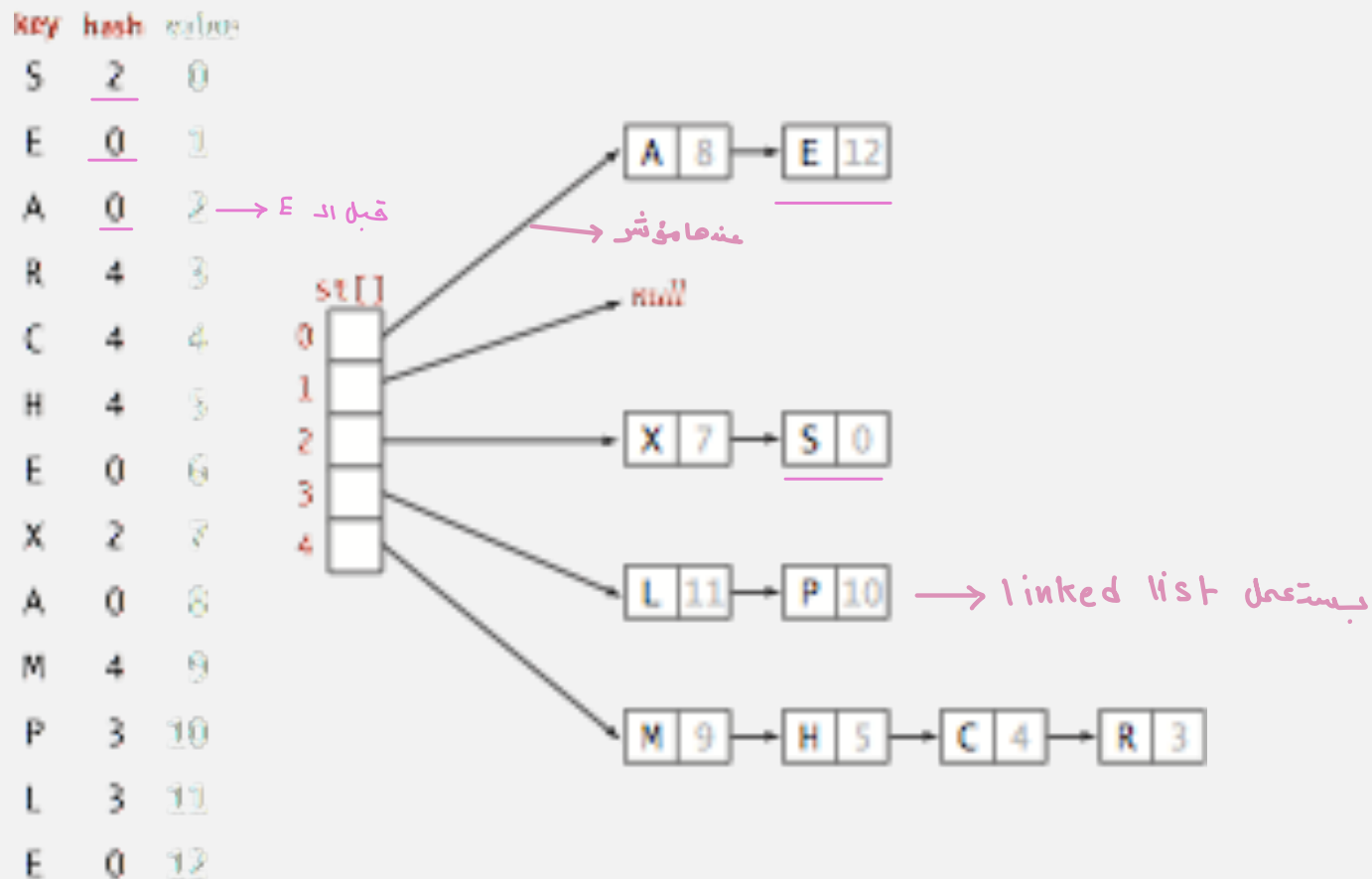
Bea	Tim	Len	Moe	Mia	Zoe	Sue	Lou	Rae	Max	Tod
0	1	2	3	4	5	6	7	8	9	10

© 2014 Goodrich, Tamassia, Godwasser Hash Tables 20

Separate-chaining symbol table

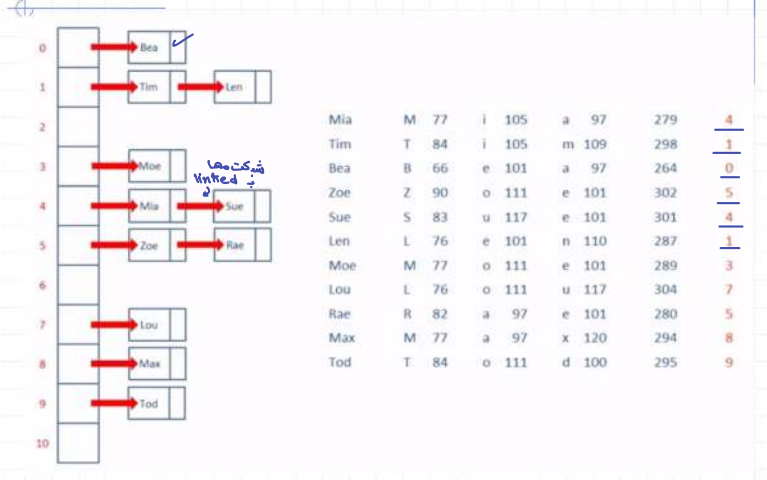
Use an array of $M < N$ linked lists. [H. P. Luhn, IBM 1953]

- Hash: map key to integer i between 0 and $M - 1$.
- Insert: put at front of i^{th} chain (if not already there).
- Search: need to search only i^{th} chain.



تمثيل تصاريح + linked list

Other way to deal with collision is **chaining**, here array has pointer to first node



★ Hash function : $O(1)$

address = key mod n

alphanumeric → أرقام وأحرف
يجمعونه أو يفرقونه

Folding method: divides key into parts then adds the parts together.
نقسم المفتاح إلى أجزاء ثم نجمعها مع بعضها
نجمعها ونجيب بأجزاء الأجزاء من array

Separate-chaining symbol table: Java implementation

```
public class SeparateChainingHashST<Key, Value>
{
    private int M = 97;           // number of chains
    private Node[] st = new Node[M]; // array of chains

    private static class Node
    {
        private Object key;
        private Object val;
        private Node next;
        ...
    }

    private int hash(Key key)
    { return (key.hashCode() & 0x7fffffff) % M; }

    public Value get(Key key) {
        int i = hash(key);
        for (Node x = st[i]; x != null; x = x.next)
            if (key.equals(x.key)) return (Value) x.val;
        return null;
    }
}
```

array doubling and
halving code omitted

no generic array creation

(declare key and value of type Object)

Separate-chaining symbol table: Java implementation

```
public class SeparateChainingHashST<Key, Value>
{
    private int M = 97;           // number of chains
    private Node[] st = new Node[M]; // array of chains

    private static class Node
    {
        private Object key;
        private Object val;
        private Node next;
        ...
    }

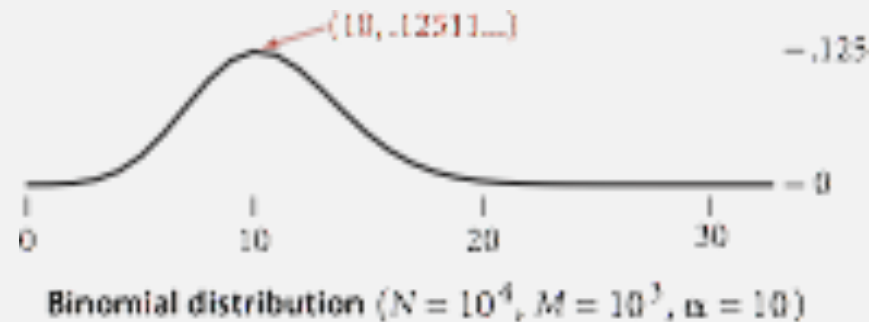
    private int hash(Key key)
    { return (key.hashCode() & 0x7fffffff) % M; }

    public void put(Key key, Value val) {
        int i = hash(key);
        for (Node x = st[i]; x != null; x = x.next)
            if (key.equals(x.key)) { x.val = val; return; }
        st[i] = new Node(key, val, st[i]);
    }
}
```

Analysis of separate chaining

Proposition. Under uniform hashing assumption, prob. that the number of keys in a list is within a constant factor of N/M is extremely close to 1.

Pf sketch. Distribution of list size obeys a binomial distribution.



Consequence. Number of probes for search/insert is proportional to N/M .
↖ equals() and hashCode()
↖ عدد المحاولات

- M too large $\Rightarrow$ too many empty chains.
- M too small $\Rightarrow$ chains too long.
- Typical choice: $M \sim N/4 \Rightarrow$ constant-time ops.

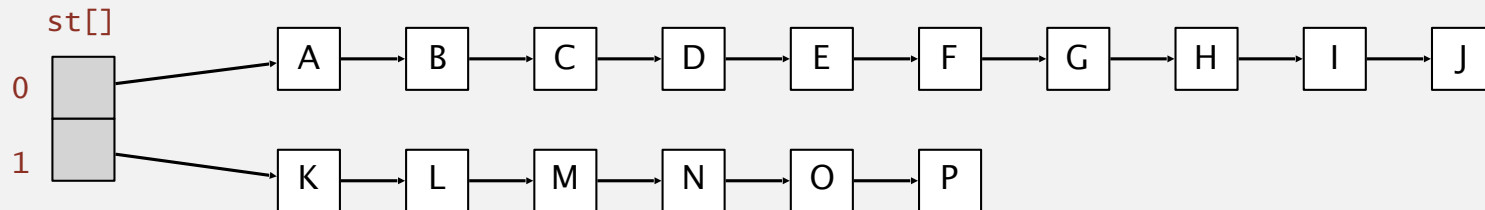
↑
M times faster than
sequential search

Resizing in a separate-chaining hash table

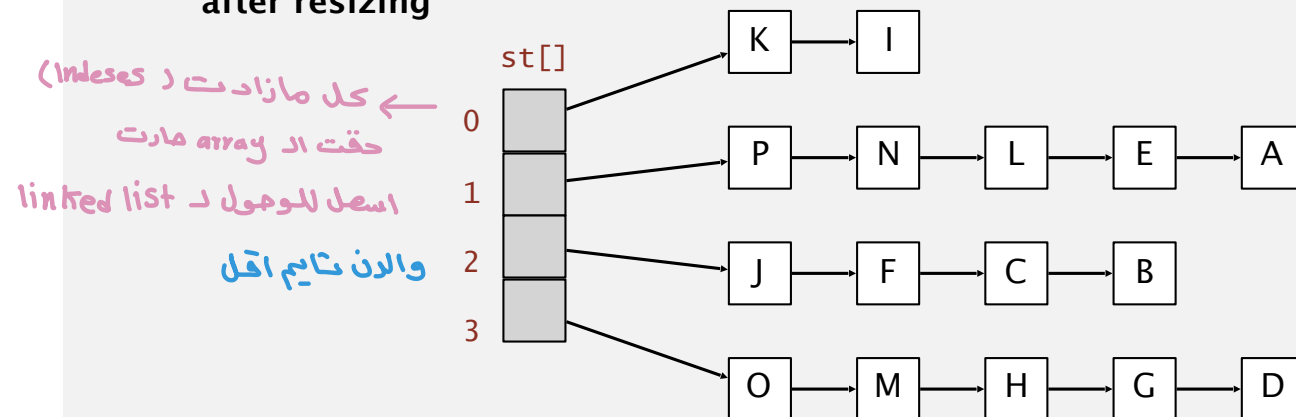
Goal. Average length of list $N / M = \text{constant}$.

- Double size of array M when $N / M \geq 8$. ← *بمغزما*
- Halve size of array M when $N / M \leq 2$. ← *بمکرمما*
- Need to rehash all keys when resizing. ← $x.\text{hashCode}()$ does not change but $\text{hash}(x)$ can change

before resizing



after resizing



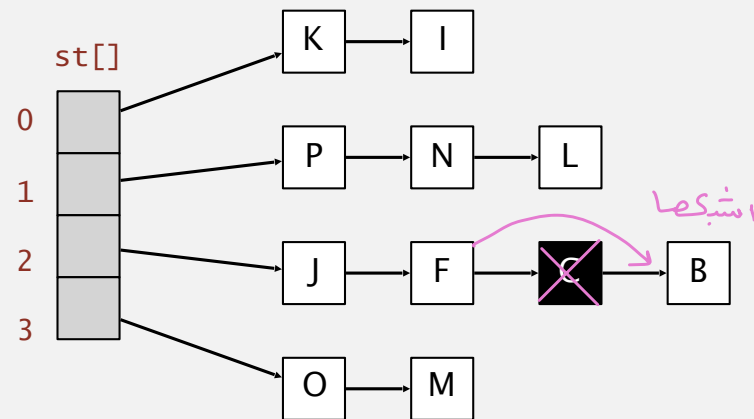
Deletion in a separate-chaining hash table

Q. How to delete a key (and its associated value)?

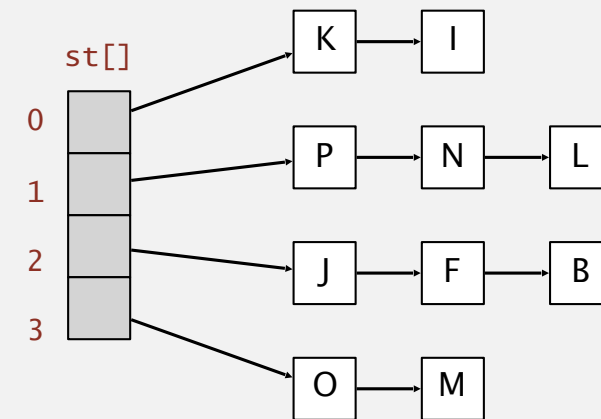
A. Easy: need only consider chain containing key.

1- نود عليه
2- بحذفه واشبك
الي بعده.

before deleting C



after deleting C



Q1. Draw the 11-entry hash table that results from using the hash function, $h(i) = (3i+5) \bmod 11$, to hash the keys 12, 44, 13, 88, 23, 94, 11, 39, 20, 16, and 5, assuming collisions are handled by chaining.

نحل فيها ال collisions
نستخدم ال linked list

$$h(12) = (3 \times 12 + 5) \bmod 11 = 8$$

$$h(44) = (3 \times 44 + 5) \bmod 11 = 5$$

$$h(13) = (3 \times 13 + 5) \bmod 11 = 0$$

$$h(88) = (3 \times 88 + 5) \bmod 11 = 5$$

$$h(23) = (3 \times 23 + 5) \bmod 11 = 8$$

$$h(94) = (3 \times 94 + 5) \bmod 11 = 1$$

$$h(11) = (3 \times 11 + 5) \bmod 11 = 5$$

$$h(39) = (3 \times 39 + 5) \bmod 11 = 1$$

$$h(20) = (3 \times 20 + 5) \bmod 11 = 10$$

$$h(16) = (3 \times 16 + 5) \bmod 11 = 9$$

$$h(5) = (3 \times 5 + 5) \bmod 11 = 9$$

0		→ [13] → ∅
1		→ [94] → [39] → ∅
2		∅
3		∅
4		∅
5		→ [44] → [88] → [11] → ∅
6		∅
7		∅
8		→ [12] → [23] → ∅
9		→ [16] → [5] → ∅
10		→ [20] → ∅

Symbol table implementations: summary

implementation	guarantee			average case			ordered ops?	key interface
	search	insert	delete	search hit	insert	delete		
sequential search (unordered list)	N	N	N	$\frac{1}{2} N$	N	$\frac{1}{2} N$		equals()
binary search (ordered array)	$\lg N$	N	N	$\lg N$	$\frac{1}{2} N$	$\frac{1}{2} N$	✓	compareTo()
BST ما فیہا ترتیب او نو ما مرتبہ	N	N	N	$1.39 \lg N$	$1.39 \lg N$	$\sqrt{N}$	✓	compareTo()
red-black BST	$2 \lg N$	$2 \lg N$	$2 \lg N$	$1.0 \lg N$	$1.0 \lg N$	$1.0 \lg N$	✓	compareTo()
separate chaining	N	N	N ↓	$3-5 *$	$3-5 *$	$3-5 *$		equals() hashCode()

عدد ال element
 ۸ ن لزم ادخل element
 element.

* under uniform hashing assumption



<http://algs4.cs.princeton.edu>

3.4 HASH TABLES

- *hash functions*
- *separate chaining*
- *linear probing* → خريفة خنيفة
- *context*

+ اذا عندي نظام بياكد
بلازم لاني ببعدها اذا خايفة
بلازم لا element واذا مو فاضيه اكل
واذا وحلت نصايه الاربي ما اوقف امتر
هذي (Prob) توخاني للبيانية .

Collision resolution: open addressing

Open addressing. [Amdahl-Boehme-Rochester-Samuel, IBM 1953]

When a new key collides, find next empty slot, and put it there.

فكرتھا : بروخ للخانه القیما ملیانہ

بروخ للی بعدھا اور علیھا
الین ما احصل اول واحدہ فاخیرہ

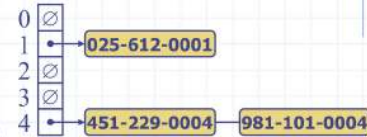
st[0]	jocularly
st[1]	<i>null</i>
st[2]	listen
st[3]	suburban
⋮	<i>null</i>
st[30000]	browsing

linear probing ($M = 30001$, $N = 15000$)

Collision Handling



- (1) Separate Chaining: let each cell in the table point to a linked list of entries that map there



- Separate chaining is simple, but requires additional memory outside the table

طريقة حل الـ Colliding :

★ Linear Probing: ديفيف جاول مكان فاضي ويرجع من البداية.

★ عدد الـ Probe : عدد المحاولات الي جاول فيهم يلاقى مكان فاضي

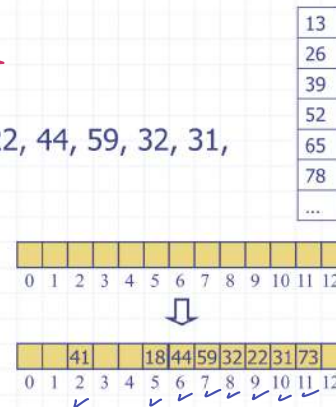
Linear Probing

- Example:

$$h(x) = x \bmod 13$$

- Insert keys 18, 41, 22, 44, 59, 32, 31, 73, in this order.

1. $18 \bmod 13 = 5$ ✓
2. $41 \bmod 13 = 2$ ✓
3. $22 \bmod 13 = 9$ ✓
4. $44 \bmod 13 = 5$ ✓
5. $59 \bmod 13 = 7$ ✓
6. $32 \bmod 13 = 6$ ✓
7. $31 \bmod 13 = 5$ ✓
8. $73 \bmod 13 = 8$ ✓



Colligen: $O(n)$ ← جوف كداد array جودر على اول مكان فاضي

bcs: $O(1)$ ← يروح حمله ماول

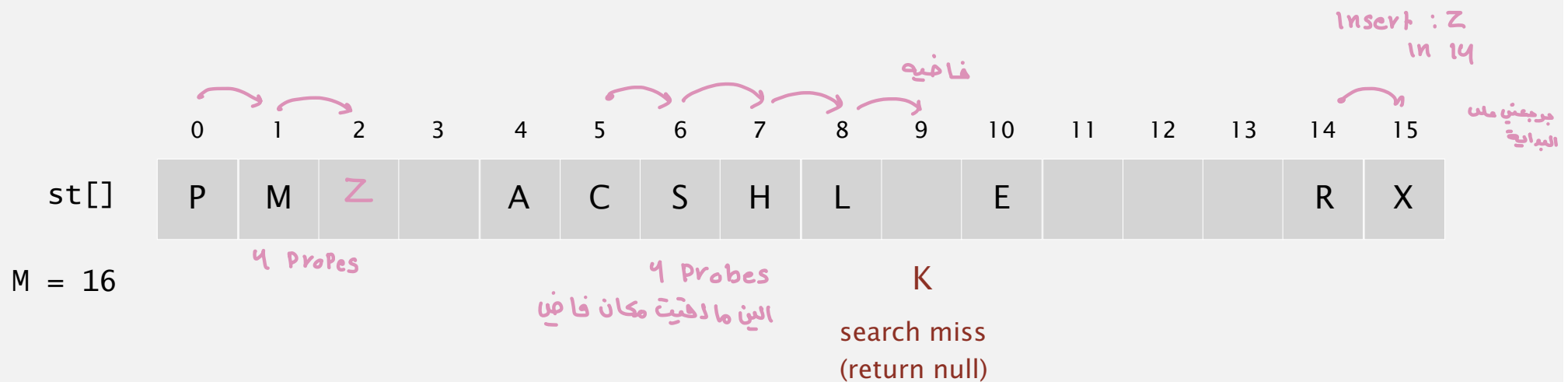
Linear-probing hash table demo

Hash. Map key to integer i between 0 and $M-1$.

Search. Search table index i ; if occupied but no match, try $i+1$, $i+2$, etc.

search K

hash(K) = 5



Q2. Do the same as above using **Linear Probing**.

||||| # of collisions? 6

$$h(12) = (3 \times 12 + 5) \bmod 11 = 8$$

$$h(44) = (3 \times 44 + 5) \bmod 11 = 5$$

$$h(13) = (3 \times 13 + 5) \bmod 11 = 0$$

$$h(88) = (3 \times 88 + 5) \bmod 11 = 5 \text{ collision}$$

$$h(29) = (3 \times 29 + 5) \bmod 11 = 8$$

$$h(94) = (3 \times 94 + 5) \bmod 11 = 1$$

$$h(11) = (3 \times 11 + 5) \bmod 11 = 5$$

$$h(39) = (3 \times 39 + 5) \bmod 11 = 1$$

$$h(20) = (3 \times 20 + 5) \bmod 11 = 10$$

$$h(16) = (3 \times 16 + 5) \bmod 11 = 9$$

$$h(5) = (3 \times 5 + 5) \bmod 11 = 9$$

		# of Probes ?	بكم مره خزلت ادور مكان فافين
0	13		16
1	94		
2	39		
3	16		
4	5		
5	44		
6	88		
7	11		
8	12		
9	23		
10	20		

عشان افوف
كم تضاد معني

خزلت مرتين
ادور مكان فافين

Linear-probing hash table summary

Hash. Map key to integer i between 0 and $M-1$.

Insert. Put at table index i if free; if not try $i+1$, $i+2$, etc.

Search. Search table index i ; if occupied but no match, try $i+1$, $i+2$, etc.

Note. Array size M **must be** greater than number of key-value pairs N .

number of element

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
st[]	P	M			A	C	S	H	L		E				R	X

$M = 16$

Linear-probing symbol table: Java implementation

```
public class LinearProbingHashST<Key, Value>
{
    private int M = 30001;
    private Value[] vals = (Value[]) new Object[M];
    private Key[] keys = (Key[]) new Object[M];

    private int hash(Key key)          { /* as before */ }

    private void put(Key key, Value val) { /* next slide */ }

    public Value get(Key key)
    {
        for (int i = hash(key); keys[i] != null; i = (i+1) % M)
            if (key.equals(keys[i]))
                return vals[i];
        return null;
    }
}
```

← array doubling and
halving code omitted

Linear-probing symbol table: Java implementation

```
public class LinearProbingHashST<Key, Value>
{
    private int M = 30001;
    private Value[] vals = (Value[]) new Object[M];
    private Key[] keys = (Key[]) new Object[M];

    private int hash(Key key) { /* as before */ }

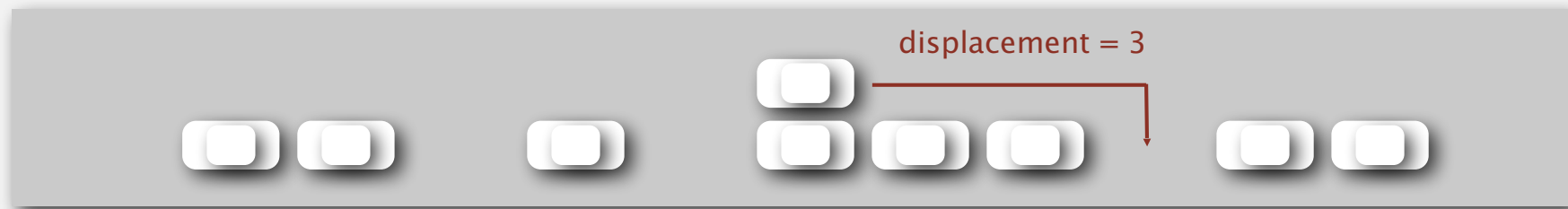
    private Value get(Key key) { /* previous slide */ }

    public void put(Key key, Value val)
    {
        int i;
        for (i = hash(key); keys[i] != null; i = (i+1) % M)
            if (keys[i].equals(key))
                break;
        keys[i] = key;
        vals[i] = val;
    }
}
```


Knuth's parking problem

Model. Cars arrive at one-way street with M parking spaces. Each desires a random space i : if space i is taken, try $i + 1, i + 2$, etc.

Q. What is mean displacement of a car?



Half-full. With $M / 2$ cars, mean displacement is $\sim 3 / 2$.

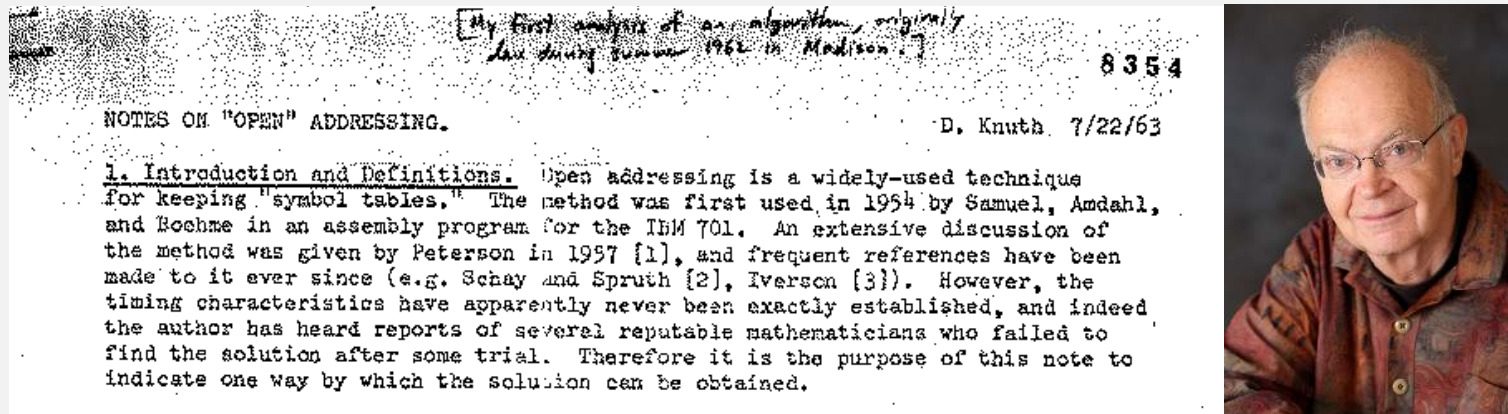
Full. With M cars, mean displacement is $\sim \sqrt{\pi M / 8}$.

Analysis of linear probing

Proposition. Under uniform hashing assumption, the average # of probes in a linear probing hash table of size M that contains $N = \alpha M$ keys is:

$$\begin{array}{ll} \sim \frac{1}{2} \left(1 + \frac{1}{1 - \alpha} \right) & \sim \frac{1}{2} \left(1 + \frac{1}{(1 - \alpha)^2} \right) \\ \text{search hit} & \text{search miss / insert} \end{array}$$

Pf.



Parameters.

- M too large $\Rightarrow$ too many empty array entries.
- M too small $\Rightarrow$ search time blows up.
- Typical choice: $\alpha = N / M \sim 1/2$. ← # probes for search hit is about 3/2
probes for search miss is about 5/2

Resizing in a linear-probing hash table

Goal. Average length of list $N / M \leq \frac{1}{2}$.

- Double size of array M when $N / M \geq \frac{1}{2}$.
- Halve size of array M when $N / M \leq \frac{1}{8}$.
- Need to rehash all keys when resizing.

before resizing

	0	1	2	3	4	5	6	7
keys[]		E	S			R	A	
vals[]		1	0			3	2	

after resizing

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
keys[]					A		S				E				R	
vals[]					2		0				1				3	

Deletion in a linear-probing hash table

Q. How to delete a key (and its associated value)?

A. Requires some care: can't just delete array entries.

1- Search for the key.

2- إذا كان (null) ما → اول مكان فارغ بيقف
بيومل للحرف اللي بيورة
بيقف عند الخانة الخاطيه.

before deleting S

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
keys[]	P	M			A	C	S	H	L		E				R	X
vals[]	10	9			8	4	0	5	11		12				3	7

delete(S)

doesn't work, e.g., if hash(H) = 4

after deleting S ?

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
keys[]	P	M			A	C		H	L		E				R	X
vals[]	10	9			8	4		5	11		12				3	7

ST implementations: summary

implementation	guarantee			average case			ordered ops?	key interface
	search	insert	delete	search hit	insert	delete		
sequential search (unordered list)	N	N	N	$\frac{1}{2} N$	N	$\frac{1}{2} N$		equals()
binary search (ordered array)	$\lg N$	N	N	$\lg N$	$\frac{1}{2} N$	$\frac{1}{2} N$	✓	compareTo()
BST	N	N	N	$1.39 \lg N$	$1.39 \lg N$	$\sqrt{N}$	✓	compareTo()
red-black BST	$2 \lg N$	$2 \lg N$	$2 \lg N$	$1.0 \lg N$	$1.0 \lg N$	$1.0 \lg N$	✓	compareTo()
separate chaining	N	N	N	$3-5 *$	$3-5 *$	$3-5 *$		equals() hashCode()
linear probing	N	N	N	$3-5 *$	$3-5 *$	$3-5 *$		equals() hashCode()

Assume the array is full

* under uniform hashing assumption



<http://algs4.cs.princeton.edu>

3.4 HASH TABLES

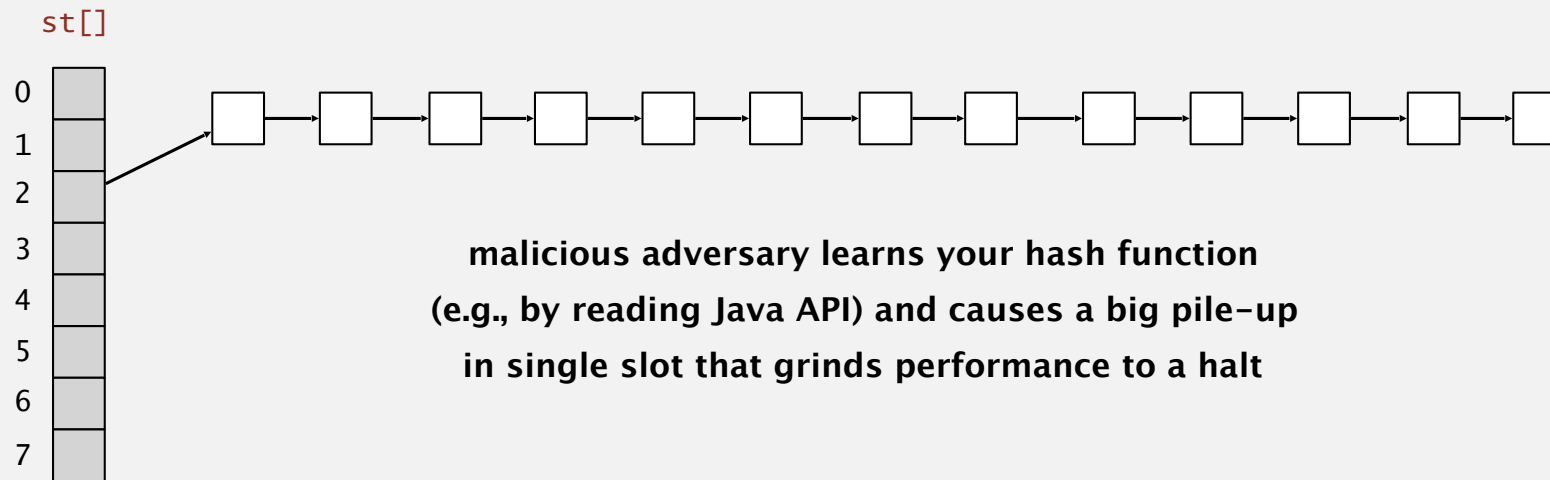
- *hash functions*
- *separate chaining*
- *linear probing*
- *context*

War story: algorithmic complexity attacks

Q. Is the uniform hashing assumption important in practice?

A. Obvious situations: aircraft control, nuclear reactor, pacemaker.

A. Surprising situations: **denial-of-service** attacks.



Real-world exploits. [Crosby-Wallach 2003]

- Bro server: send carefully chosen packets to DOS the server, using less bandwidth than a dial-up modem.
- Perl 5.8.0: insert carefully chosen strings into associative array.
- Linux 2.4.20 kernel: save files with carefully chosen names.

Algorithmic complexity attack on Java

Goal. Find family of strings with the same hash code.

Solution. The base-31 hash code is part of Java's string API.

key	hashCode()
"Aa"	2112
"BB"	2112

key	hashCode()
"AaAaAaAa"	-540425984
"AaAaAaBB"	-540425984
"AaAaBBaA"	-540425984
"AaAaBBBB"	-540425984
"AaBBaAaA"	-540425984
"AaBBaABB"	-540425984
"AaBBBBaA"	-540425984
"AaBBBBBB"	-540425984

key	hashCode()
"BBaAaAaA"	-540425984
"BBaAaABB"	-540425984
"BBaABBAa"	-540425984
"BBaBBBBB"	-540425984
"BBBBaAaA"	-540425984
"BBBBaABB"	-540425984
"BBBBBBaA"	-540425984
"BBBBBBBB"	-540425984

2^N strings of length $2N$ that hash to same value!

Diversion: one-way hash functions

One-way hash function. "Hard" to find a key that will hash to a desired value (or two keys that hash to same value).

Ex. MD4, MD5, SHA-0, SHA-1, SHA-2, WHIRLPOOL, RIPEMD-160,

known to be insecure

```
String password = args[0];  
MessageDigest sha1 = MessageDigest.getInstance("SHA1");  
byte[] bytes = sha1.digest(password);  
  
/* prints bytes as hex string */
```

Applications. Digital fingerprint, message digest, storing passwords.

Caveat. Too expensive for use in ST implementations.

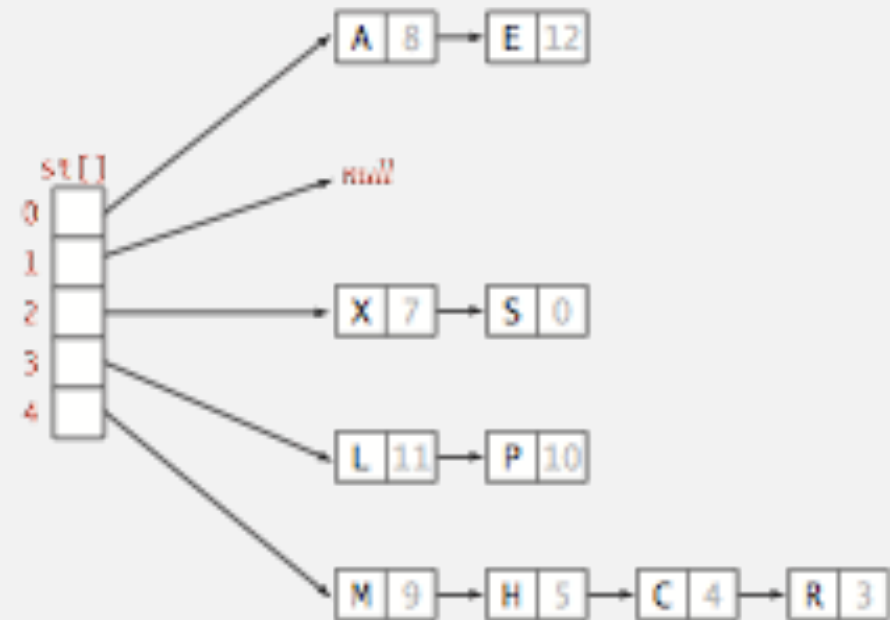
Separate chaining vs. linear probing

Separate chaining.

- Performance degrades gracefully.
- Clustering less sensitive to poorly-designed hash function.

Linear probing.

- Less wasted space.
- Better cache performance.



	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
keys[]	P	M			A	C	S	H	L		E				R	X
vals[]	10	9			8	4	0	5	11		12				3	7

Hashing: variations on the theme

Many improved versions have been studied.

Two-probe hashing. [separate-chaining variant]

- Hash to two positions, insert key in shorter of the two chains.
- Reduces expected length of the longest chain to $\log \log N$.

Two function : Primary → الأساسية → Index المجلد
Secondary : ثانوية → يعطيني كم مرة انقل

Double hashing. [linear-probing variant]

- Use linear probing, but skip a variable amount, not just 1 each time.
- Effectively eliminates clustering.
- Can allow table to become nearly full.
- More difficult to implement delete.

Cuckoo hashing. [linear-probing variant]

- Hash key to two positions; insert key into either position; if occupied, reinsert displaced key into its alternative position (and recur).
- Constant worst-case time for search.



Use double hashing to enter the following keys in an array of size 13.

98, 33, 36, 96, 77, 11, 27, 84, 66, 58

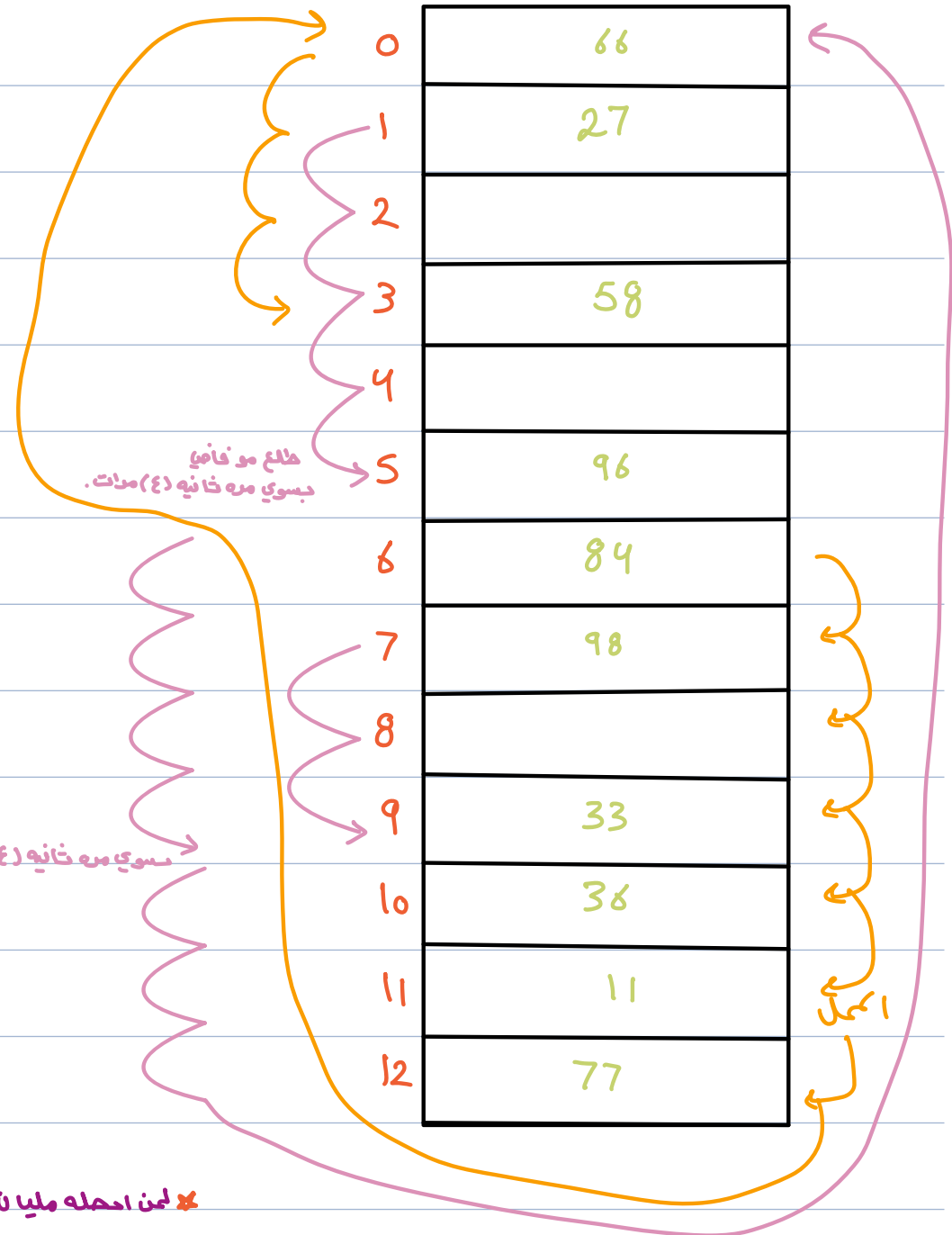
Use primary hash function $h(k) = k \% 13$.

Secondary hash function $s(k) = 7 - k \% 7$. → حاله لو فيه صفر ارجع لاني بعدها .

	$h(k) = k \% 13$	$S(k) = 7 - k \% 7$ →	ما ننتجها الين ما نحتاج لها
98	7	7	
33	7	2	
36	10	6	
96	5	2	
77	12	7	
11	11	3	
27	1	1	
84	6	7	
66	1	4	
58	6	5	

لكن احمله مليا ان بمشي بنفس الزيادة

منه ثانيه الين ما احمل خانه فاخيرة .



Use double hashing to enter the following keys in an array of size 13.

98, 33, 36, 96, 77, 11, 27, 84, 66, 58

Use primary hash function $h(k) = k \% 13$.

Secondary hash function $s(k) = 7 - k \% 7$.

مستخدمها جدول
ما عندي تعارض
لو عندي
تعارض

$$h(k) = k \% 13$$

$$s(k) = 7 - k \% 7$$

ما نحتاجه الين
ما نحتاجه لسا

	$h(k) = k \% 13$	$s(k) = 7 - k \% 7$
98	7	7
33	7	2
36	10	6
96	5	2
77	12	7
11	11	3
27	1	1
84	6	7
66	1 تعارض	4
58	6 تعارض	5

لان بنزل مريت عكس ال (5)
بنزل ثلاث مرات

اخذ هذا

بناخذ الي دخت اكثر

+ بناخذ ال فيه Prob أقل

0	
1	27
2	33
3	
4	66
5	96
6	84
7	98
8	58
9	
10	36
11	11
12	77

Double hashing.

Suppose you are asked to insert the following keys in a hash table of size 13

19, 41, 22, 5, 13, 44, 26, 57

Given hash functions:

$H1(x) = x \text{ mode } 13$ and $H2(x) = 7 - x \text{ mode } 7$

- Use linear Probing to solve collision using $H1(x)$
- Use Chaining to solve collision using $H1(x)$
- Use Double hash where $h(x) = H1(x) + jH2(x)$
- Count the number of collisions and number of probes for each method
- Compare the methods. Which one has better performance cording the # of collisions and probes.

Solution of use linear Probing to Solve collision using $H1(x)$:

Counts of Probes			
1 - $19 = 19 \div 13 = 6$	0		
2 - $41 = 41 \div 13 = 2$	0	7 - $44 = 44 \div 13 = 5$	3
3 - $22 = 22 \div 13 = 9$	0	8 - $26 = 26 \div 13 = 0$	1
4 - $5 = 5 \div 13 = 5$	0	9 - $57 = 57 \div 13 = 5$	5
5 - $7 = 7 \div 13 = 7$	0		
6 - $13 = 13 \div 13 = 0$	0		

0	13
1	26
2	41
3	
4	
5	5
6	19
7	7
8	44
9	22
10	57
11	
12	

عدد التصادمات
Counts of collisions: 3 Item

Counts of Probes = 9 Probes

Solution of use linear Probing to Solve collision using $H1(x)$:

	Counts of Probes ↓		
1- $19 = 19 \div 13 = 6$	0		
2- $41 = 41 \div 13 = 2$	0	7- $44 = 44 \div 13 = 5$ → هنا لأن اوريدي مجوزة يتروح إلى بعدهما التي ما نلاقي مكان فاضي ونخذ	3
3- $22 = 22 \div 13 = 9$	0	8- $26 = 26 \div 13 = 0$	1
4- $5 = 5 \div 13 = 5$	0	9- $57 = 57 \div 13 = 5$	5
5- $7 = 7 \div 13 = 7$	0		
6- $13 = 13 \div 13 = 0$	0		

عدد التصادمات

Counts of collisions: 3 Item

No of Probes = 0

0	13 → 26	
1	26	
2	41	
3		
4		
5	5 → 44 → 57	
6	19	
7	7	
8	44	
9	22	
10	57	
11		
12		

Solution of use linear Probing to Solve collision using $h(x) = H_1(x) + jH_2(x)$

$H_1(x)$

↘

$H_2(x) = 7 - x \text{ mod } 7$

↘

$H(x) = h_1(x) + jh_2(x)$

↘

13

$$1 - 19 = 19 \div 7 = 2$$

$$6 + (0) \times 2 = 6$$

o

26

$$2 - 41 = 41 \div 7 = 1$$

$$2 + (0) \times 1 = 2$$

o

41

$$3 - 22 = 22 \div 7 = 6$$

$$9 + (0) \times 6 = 9$$

o

$$4 - 5 = 5 \div 7 = 2$$

$$5 + (0) \times 2 = 5$$

o

$$5 - 7 = 7 \div 7 = 7$$

$$7 + (0) \times 7 = 7$$

o

5

$$6 - 13 = 13 \div 7 = 1$$

$$0 + (0) \times 1 = 0$$

o

19

$$7 - 44 = 44 \div 7 = 5$$

$$5 + (0) \times 5 = 5$$

l

$$5 + (1) \times 5 = 10$$

7

$$8 - 26 = 26 \div 7 = 2$$

$$0 + (0) \times 2 = 0 \rightarrow \text{لا يوجد حجرة}$$

2

$$0 + (2) \times 2 = 4$$

44

$$9 - 57 = 57 \div 7 = 6$$

$$5 + (0) \times 6 = 5$$

l

$$5 + (1) \times 6 = 11$$

22

57

0	13
1	
2	41
3	
4	26
5	5
6	19
7	7
8	
9	22
10	44

عدد التصادمات
Counts of collisions: 3 Item

number of probes = 4 Probes

11	57
12	